

Projeto 4

Cálculo de Folha de Pagamento

Comparação entre paradigmas: **imperativo**, **orientado a objetos**, **funcional** e **lógico**

UESPI · Paradigmas de Linguagens de Programação · 2026.1

Integrantes: Filipe Genesis | Alex Soares | Francisco Luan

O problema

Implementar o mesmo cálculo sob quatro formas de pensar

Objetivo do sistema

Entrada

- Nome do funcionário
- Salário bruto
- Número de dependentes
- Horas extras

Saída

- Valor das horas extras
- INSS
- IRRF
- Salário líquido
- Total, maior e menor líquido

O foco do trabalho não é apenas calcular corretamente, mas comparar como cada paradigma organiza regras, dados e agregações.

Regras principais

Fórmulas

```
valor_hora = salário_bruto / 180
extras = valor_hora × 1,5 × horas
base_irrf = bruto + extras - INSS - dependentes×189,59
líquido = bruto + extras - INSS - IRRF
```

Faixas simplificadas

Imposto	Faixas usadas
INSS	7,5%, 9%, 12%, 14%
IRRF	isento, 7,5%, 15%, 22,5%, 27,5%

Premissa: as quatro linguagens usam as mesmas tabelas e só formatam valores monetários com 2 casas na saída.

Caso de teste obrigatório

Funcionário	Bruto	Extras	INSS	IRRF	Líquido
Ana	4500.00	300.00	630.00	852.93	3317.07
Bruno	2200.00	0.00	198.00	0.00	2002.00
Carla	1300.00	43.33	97.50	0.00	1245.83

Total da folha

6564.90

Maior líquido

Ana

Menor líquido

Carla

Quatro paradigmas

A mesma regra, quatro arquiteturas diferentes

Visão geral das soluções

Paradigma	Linguagem	Ideia central	LOC
Imperativo	C	structs, arrays, loops e acumuladores	85
Orientado a objetos	Java	entidades, estratégias e encapsulamento	102
Funcional	Haskell	funções puras e transformação de listas	87
Lógico	Prolog	fatos, regras e relações	71

LOC = linhas não vazias e não comentadas da implementação principal.

Imperativo — C

Como pensa o problema

- Dados em `structs`
- Faixas em arrays
- Busca por alíquota com `for`
- Agregados com acumuladores

Ponto forte

Fluxo explícito e fácil de depurar passo a passo.

Fricção

Mais responsabilidade manual sobre índices, arrays e mutação.

```
static double buscar_aliquota(double valor,
                              const Faixa faixas[],
                              size_t quantidade) {
    for (size_t i = 0; i < quantidade; i++) {
        if (valor ≤ faixas[i].limite)
            return faixas[i].aliquota;
    }
    return faixas[quantidade - 1].aliquota;
}

for (size_t i = 0; i < quantidade; i++) {
    Resultado atual = calcular_funcionario(
        &funcionarios[i]);
    total += atual.liquido;
}
```


Orientado a objetos — Java

Como pensa o problema

- `Funcionario` e `ResultadoFolha`
- `Faixa` como objeto de domínio
- `RegraDesconto` como Strategy
- `CalculadoraFolha` coordena o cálculo

Ponto forte

Boa extensibilidade quando regras mudam.

Fricção

Mais estrutura do que o problema mínimo exige.

```
interface RegraDesconto {
    double calcular(double base);
}

class RegraPorFaixas implements RegraDesconto {
    public double calcular(double base) {
        double aliquota = faixas.stream()
            .filter(f -> f.contem(base))
            .findFirst()
            .orElseThrow()
            .aliquota();
        return base * aliquota;
    }
}
```

Funcional — Haskell

Como pensa o problema

- Dados imutáveis
- Funções puras
- `map` transforma funcionários em resultados
- `sum`, `maximumBy`, `minimumBy` calculam agregados

Ponto forte

O domínio é naturalmente uma transformação de lista.

Fricção

Exige pensar em composição, não em atualização de variáveis.

```
aliquota [] _ = error "Tabela de faixas vazia"
aliquota ((limite, taxa):resto) valor
    | valor <= limite = taxa
    | otherwise = aliquota resto valor

calcularFuncionario f =
    ResultadoFolha nome bruto extras inss irrf liquido
  where
    bruto = salarioBruto f
    extras = valorExtras f
    inss = bruto * aliquota faixasINSS bruto
    base = max 0.0
        (bruto + extras - inss - deducaoDependentes f)
    irrf = base * aliquota faixasIRRF base
    liquido = bruto + extras - inss - irrf
```

Lógico — Prolog

Como pensa o problema

- Faixas como fatos
- Cálculos como predicados
- `folha/2` relaciona lista de entrada e resultados
- Sem estado mutável simulado

Ponto forte

Regras de faixas ficam declarativas e compactas.

Fricção

Aritmética depende de instanciação e ordem dos predicados.

```
faixa_inss(1412.00, 0.075).
faixa_inss(2666.00, 0.090).
faixa_inss(4000.00, 0.120).
faixa_inss(infinito, 0.140).

aliquota(Tipo, Valor, Aliquota) :-
    faixa(Tipo, Limite, Aliquota),
    dentro_da_faixa(Valor, Limite),
    !.

folha(Funcionarios, Resultados) :-
    maplist(calcular_funcionario,
            Funcionarios,
            Resultados).
```

Comparação

O que mudou ao trocar o paradigma?

Representação das faixas

C

Arrays de `Faixa`

Java

Objetos `Faixa` dentro de estratégias

Haskell

Listas de pares `(limite, taxa)`

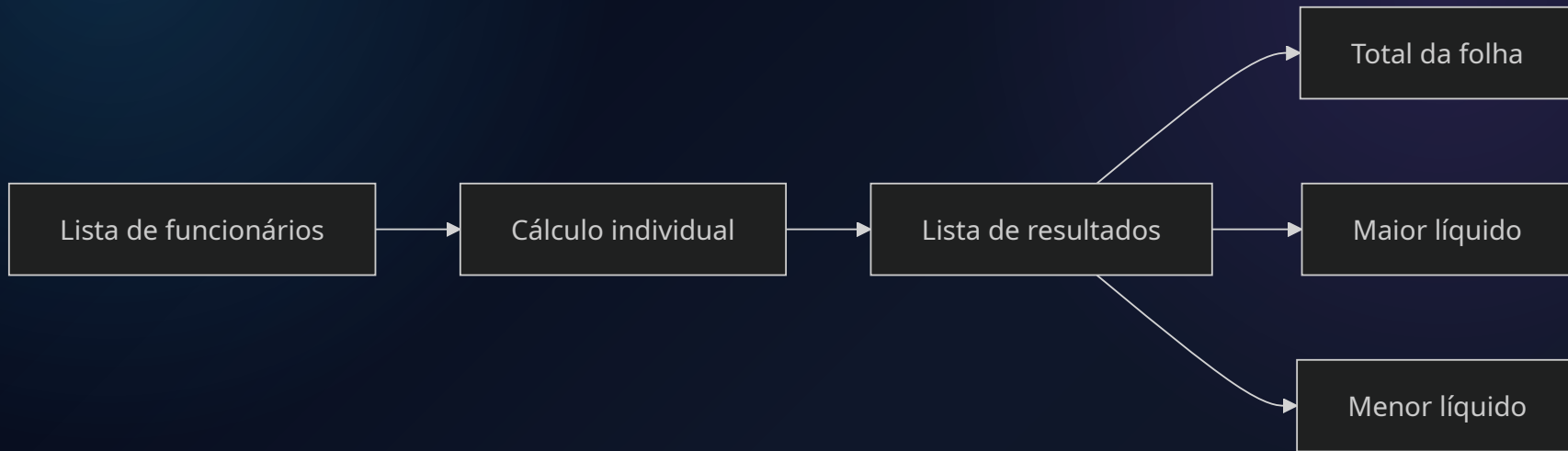
Prolog

Fatos `faixa\_inss/2` e `faixa\_irrf/2`

Insight

Adicionar uma nova faixa é simples em todas as versões, mas o impacto arquitetural é diferente: em Java a alteração fica encapsulada na estratégia; em Prolog vira um novo fato; em C e Haskell é uma nova entrada na tabela.

Estado e processamento da lista



Mutação explícita

- C usa acumuladores durante o loop
- Fácil acompanhar, mas exige disciplina

Transformação declarativa

- Haskell e Prolog separam cálculo individual e agregação
- Java usa streams para aproximar essa ideia

O que cada paradigma evidenciou

Paradigma	Onde brilhou	Onde trouxe fricção
C	Controle direto do fluxo	Controle manual de estado
Java	Extensibilidade e encapsulamento	Verbosidade estrutural
Haskell	Cálculo puro e agregações	Curva de aprendizado funcional
Prolog	Regras compactas e declarativas	Aritmética não é tão natural

A solução mais curta não foi automaticamente a mais simples: Prolog teve menos LOC, mas exige familiaridade com unificação, corte e avaliação aritmética.

Resultados de execução

Todas as implementações produziram valores equivalentes.

```
Total da folha: 6564.90  
Maior salário líquido: Ana (3317.07)  
Menor salário líquido: Carla (1245.83)
```

C
OK

Java
OK

Haskell
OK

Prolog
OK

Conclusão

Não existe paradigma universalmente melhor — existe adequação ao problema

Síntese final

Para este domínio

O paradigma **funcional** ficou especialmente adequado porque o problema é uma transformação de dados:

```
funcionários → resultados → agregados
```

Para evolução futura

A abordagem **orientada a objetos** seria vantajosa se novas regras, categorias ou políticas de desconto surgissem com frequência.

O exercício mostrou que trocar de paradigma muda mais do que a sintaxe: muda a arquitetura, o controle de estado e a forma de justificar decisões técnicas.

Obrigado!
Perguntas?

Projeto 4 — Cálculo de Folha de Pagamento
Paradigmas de Linguagens de Programação